

MINI PROJECT REPORT

On

“Colorizing Old Black & White Images using Deep Learning”

A Report Submitted for a mini project for: Laboratory Practice-V in 8th
Semester Computer Engineering.

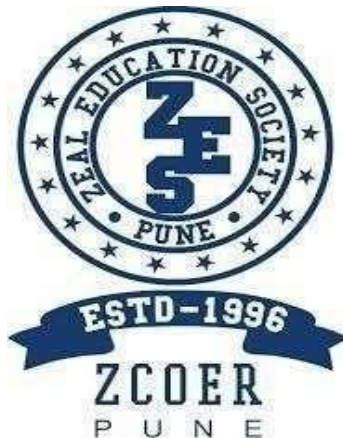
Fourth Year (COMPUTER ENGINEERING)

Academic Year 2025-26

Submitted by-

Jenil Girish Rathod

Roll No.: B22009 Div.: B



Zeal Education Society's

Zeal College of Engineering & Research Narhe,

Pune – 411041

Department of Computer Engineering

Zeal Education Society's
Zeal College of Engineering & Research Department of Computer
Engineering



CERTIFICATE

Certified that the project entitled “**Colorizing Old Black & White Images using Deep Learning**” is a bona fide work carried out by **Jenil Girish Rathod (B22009)**. It is certified that all corrections/suggestions indicated for Internal Assignment have been incorporated in the report. The project report has been approved as it satisfies the academic requirements in respect of Project work prescribed for the Bachelor of Engineering Degree.

Prof. Reesma
Project Guide

Prof. A. V. Mote
H. O. D

ACKNOWLEDGEMENT

We take this opportunity to thank our project guide **Prof. Reesma** mam and Head of the department **Prof. A. V. Mote** mam for their valuable guidance and for providing all the necessary facilities, which were indispensable in the completion of this project report. We are also thankful to all the staff members of Computer Engineering Department for their valuable time, support, comments, suggestions and persuasion. We would also like to thank the institute for providing the required facilities, Internet access and important books.

INDEX

Sr. No	CONTENT	Page No.
1.	Abstract	5
2.	Software Requirement	6
3.	Introduction	7
4.	Problem Statement	8
5.	Objective And Outcome	8
6.	Implementation Code	9
7.	Test Cases	24
8.	Output	25
9.	Conclusion	27
10.	References	27

ABSTRACT

This project presents the implementation of an automated image colourization system that converts old black-and-white images into realistic coloured images using deep learning techniques. Image colourization is a challenging task that requires understanding image context and assigning plausible colours to grayscale inputs. Traditional manual methods are time-consuming and require expertise, making them inefficient for large-scale restoration. This project leverages deep learning models to automate the process and produce visually meaningful results.

To achieve this, the system utilizes two approaches: an OpenCV DNN-based model and a custom U-Net architecture implemented using PyTorch. The input image is converted into LAB colour space, where the luminance (L) channel is used as input, and the model predicts the chrominance (a, b) channels. These predicted channels are then combined with the original L channel to reconstruct the final colour image. A user-friendly interface is developed using Streamlit, allowing users to upload images, select models, and visualize outputs interactively.

The proposed system demonstrates effective colourization with improved visual quality and flexibility through model selection and parameter tuning. It also supports additional features such as contrast enhancement, feature map visualization, and downloadable outputs. This project highlights the practical application of deep learning in image restoration and showcases how modern AI techniques can enhance historical photographs efficiently.

SOFTWARE AND HARDWARE REQUIREMENT

Software Requirements:

1. **Programming Language:** Python 3.x
2. **Framework / UI:** Streamlit (for web-based interface)
3. **Deep Learning Libraries:** PyTorch (for U-Net model implementation)
4. **Image Processing Libraries:** OpenCV (DNN module for colorization), NumPy
5. **Additional Libraries Used:** Pandas Pillow
6. **IDE / Editor:** VS Code / PyCharm / Jupyter Notebook
7. **Operating System:** Windows / Linux / macOS

Hardware Requirements

1. **Processor (CPU):**
 - **Minimum:** Intel Core i3 (or equivalent) or higher
 - **Recommended:** Intel Core i5/i7 or equivalent AMD processor
2. **Memory (RAM):**
 - **Minimum:** 4 GB of RAM
 - **Recommended:** 8 GB of RAM or more
3. **Storage:**
 - **Minimum:** 2 GB of free disk space
 - **Recommended:** 5 GB of free disk space or more
4. **Display:**
 - **Minimum:** 1024x768 screen resolution
 - **Recommended:** 1920x1080 screen resolution (Full HD)
5. **Graphics Processing Unit (GPU):**
 - **Minimum:** NVIDIA GPU with CUDA support
 - **Recommended:** NVIDIA GTX/RTX Series

INTRODUCTION

In the modern era of digital media and artificial intelligence, image processing has become an essential field for enhancing visual data and restoring historical content. One important application is image colorization, which involves converting grayscale images into realistic colored images. While converting color images to grayscale is straightforward, the reverse process is complex as it requires predicting appropriate colors based on context, texture, and semantics within the image.

Traditional colorization methods rely on manual editing, which is time-consuming and requires significant artistic skill. With the advancement of deep learning, Convolutional Neural Networks (CNNs) have enabled automatic colorization by learning patterns from large datasets. These models analyze image features such as edges, objects, and surroundings to generate plausible color distributions. This project utilizes such deep learning techniques to automate the colorization process efficiently.

The system implements two approaches for colorization: an OpenCV DNN-based model and a custom U-Net architecture built using PyTorch. The input image is processed in LAB color space, where the luminance (L) channel is used as input and the model predicts chrominance (a, b) channels. These channels are combined to reconstruct the final colored image. This approach ensures that structural details are preserved while adding realistic color information.

A user-friendly web interface is developed using Streamlit, enabling users to upload grayscale images, select models, and visualize results instantly. The system also provides additional features such as contrast enhancement, feature map visualization, and downloadable outputs. Furthermore, it supports execution on both CPU and GPU, ensuring flexibility and compatibility across different environments.

This project demonstrates how deep learning can be effectively applied to real-world problems such as restoring old photographs and enhancing visual quality. It highlights the potential of artificial intelligence in image processing and provides practical insight into model implementation, system design, and user interaction.

PROBLEM STATEMENT

Traditional black-and-white images lack visual detail and realism, making it difficult to interpret historical or old photographs effectively. Manual colorization is time-consuming, requires artistic expertise, and is not scalable for large datasets. Therefore, there is a need to develop an automated system that can accurately and efficiently colorize grayscale images using deep learning techniques.

OBJECTIVE

- To develop an automated system for colorizing black-and-white images
- To implement deep learning models (U-Net and OpenCV DNN) for color prediction
- To utilize LAB color space for effective image reconstruction
- To build an interactive web application using Streamlit
- To enhance image quality using contrast and color adjustments
- To provide real-time visualization and downloadable outputs
- To support both CPU and GPU execution for flexibility

OUTCOME

- Successful conversion of grayscale images into realistic colored images
- Improved visual quality and restoration of old photographs
- Efficient use of deep learning for automatic color prediction
- Interactive web-based interface for user input and output visualization
- Support for multiple models and adjustable color parameters
- Reliable system with accurate and consistent results
- Demonstration of practical application of AI in image processing

IMPLEMENTATION CODE

Technologies Used:

- Frontend: Streamlit
- Backend: Python, PyTorch, OpenCV, NumPy, Pandas, Pillow (PIL)

- **Backend Logic**

```
from pathlib import Path
from typing import Dict, Tuple
```

```
import torch
import torch.nn as nn
```

```
class DoubleConv(nn.Module):
    """Two stacked conv blocks used throughout U-Net."""

    def __init__(self, in_channels: int, out_channels: int) -> None:
        super().__init__()
        self.block = nn.Sequential(
            nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_channels, out_channels, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.block(x)

class Down(nn.Module):
    def __init__(self, in_channels: int, out_channels: int) -> None:
        super().__init__()
        self.pool_conv = nn.Sequential(nn.MaxPool2d(2), DoubleConv(in_channels,
out_channels))

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.pool_conv(x)

class Up(nn.Module):
    def __init__(self, in_channels: int, out_channels: int) -> None:
        super().__init__()
        self.up = nn.Upsample(scale_factor=2, mode="bilinear", align_corners=True)
        self.conv = DoubleConv(in_channels, out_channels)
```

```

def forward(self, x1: torch.Tensor, x2: torch.Tensor) -> torch.Tensor:
    x1 = self.up(x1)
    diff_y = x2.size(2) - x1.size(2)
    diff_x = x2.size(3) - x1.size(3)
    x1 = nn.functional.pad(x1, [diff_x // 2, diff_x - diff_x // 2, diff_y // 2, diff_y - diff_y
// 2])
    x = torch.cat([x2, x1], dim=1)
    return self.conv(x)

class UNetColorizer(nn.Module):
    """U-Net for colorization: L channel -> (a, b) channels."""

    def __init__(self, in_channels: int = 1, out_channels: int = 2, base_channels: int = 32) -
> None:
        super().__init__()
        self.inc = DoubleConv(in_channels, base_channels)
        self.down1 = Down(base_channels, base_channels * 2)
        self.down2 = Down(base_channels * 2, base_channels * 4)
        self.down3 = Down(base_channels * 4, base_channels * 8)

        self.up1 = Up(base_channels * 8 + base_channels * 4, base_channels * 4)
        self.up2 = Up(base_channels * 4 + base_channels * 2, base_channels * 2)
        self.up3 = Up(base_channels * 2 + base_channels, base_channels)

        self.out_conv = nn.Conv2d(base_channels, out_channels, kernel_size=1)

        # Default warm-tone bias so non-pretrained inference remains visually meaningful.
        nn.init.zeros_(self.out_conv.weight)
        nn.init.constant_(self.out_conv.bias[0:1], 0.03)
        nn.init.constant_(self.out_conv.bias[1:2], 0.08)

    def forward(self, x: torch.Tensor, return_features: bool = False):
        x1 = self.inc(x)
        x2 = self.down1(x1)
        x3 = self.down2(x2)
        x4 = self.down3(x3)

        x = self.up1(x4, x3)
        x = self.up2(x, x2)
        x = self.up3(x, x1)

        out = torch.tanh(self.out_conv(x))

        if not return_features:
            return out

        features = {
            "enc1": x1,

```

```

        "enc2": x2,
        "enc3": x3,
        "bottleneck": x4,
    }
    return out, features

def load_unet_model(weights_path: Path, device: torch.device) -> Tuple[UNetColorizer,
bool]:
    """Load U-Net model and optional trained checkpoint."""
    model = UNetColorizer().to(device)
    loaded_pretrained = False

    if weights_path.exists():
        state = torch.load(weights_path, map_location=device)
        if isinstance(state, dict) and "state_dict" in state:
            state = state["state_dict"]
            model.load_state_dict(state, strict=True)
            loaded_pretrained = True

    model.eval()
    return model, loaded_pretrained

def summarize_model(model: nn.Module) -> str:
    total_params = sum(p.numel() for p in model.parameters())
    trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
    return (
        f"Model: UNetColorizer\n"
        f"Total parameters: {total_params:,}\n"
        f"Trainable parameters: {trainable_params:,}\n"
        f"Input: (B, 1, H, W) grayscale L channel\n"
        f"Output: (B, 2, H, W) predicted a,b channels"
    )

from pathlib import Path
from typing import Dict, Optional

import urllib.request
import urllib.error
import cv2
import numpy as np
import torch

from model import UNetColorizer

def load_opencv_dnn_colorization_model(model_dir: str) -> cv2.dnn_Net:
    """Load OpenCV colorization DNN from local model files."""
    model_root = Path(model_dir)

```

```

model_path = model_root / "colorization_release_v2.caffemodel"
proto_path = model_root / "colorization_deploy_v2.prototxt"
pts_path = model_root / "pts_in_hull.npy"

# Return None instead of raising so the caller can handle missing files gracefully.
if not model_path.exists() or not proto_path.exists() or not pts_path.exists():
    return None

net = cv2.dnn.readNetFromCaffe(str(proto_path), str(model_path))
pts_in_hull = np.load(str(pts_path)).transpose().reshape(2, 313, 1, 1)

net.getLayer(net.getLayerId("class8_ab")).blobs = [pts_in_hull.astype(np.float32)]
net.getLayer(net.getLayerId("conv8_313_rh")).blobs = [np.full((1, 313), 2.606,
dtype=np.float32)]

return net

def models_exist(model_dir: str) -> bool:
    """Check whether the required OpenCV DNN model files exist in model_dir."""
    model_root = Path(model_dir)
    return (
        (model_root / "colorization_release_v2.caffemodel").exists()
        and (model_root / "colorization_deploy_v2.prototxt").exists()
        and (model_root / "pts_in_hull.npy").exists()
    )

def save_uploaded_model_files(
    prototxt_file, caffemodel_file, pts_file, model_dir: str
) -> None:
    """Save uploaded model files (Streamlit UploadedFile) into model_dir."""
    model_root = Path(model_dir)
    model_root.mkdir(parents=True, exist_ok=True)

    if prototxt_file is not None:
        prototxt_path = model_root / "colorization_deploy_v2.prototxt"
        with open(prototxt_path, "wb") as f:
            f.write(prototxt_file.getbuffer())

    if caffemodel_file is not None:
        caffemodel_path = model_root / "colorization_release_v2.caffemodel"
        with open(caffemodel_path, "wb") as f:
            f.write(caffemodel_file.getbuffer())

    if pts_file is not None:
        pts_path = model_root / "pts_in_hull.npy"
        with open(pts_path, "wb") as f:
            f.write(pts_file.getbuffer())

```

```

# Candidate download URLs. prototxt and pts are generally available; caffemodel may
require mirrors.
DEFAULT_MODEL_URLS = {
    "prototxt": [

        "https://raw.githubusercontent.com/richzhang/colorization/caffe/colorization/models/color
        ization_deploy_v2.prototxt",
        ],
    "pts": [

        "https://raw.githubusercontent.com/richzhang/colorization/caffe/colorization/resources/pts
        _in_hull.npy",
        ],
    "caffemodel": [
        "https://github.com/dath1s/colorizer/blob/main/colorization_release_v2.caffemodel",

        "https://github.com/richzhang/colorization/raw/caffe/colorization/models/colorization_re
        lease_v2.caffemodel",

        "https://raw.githubusercontent.com/richzhang/colorization/caffe/colorization/models/color
        ization_release_v2.caffemodel",

        "http://eecs.berkeley.edu/~rich.zhang/projects/2016_colorization/files/demo_v2/colorizati
        on_release_v2.caffemodel",
        ],
    }

def _download_file(url: str, dest: Path, timeout: int = 60) -> None:
    """Download a single file to dest. Raises RuntimeError on failure."""
    try:
        urllib.request.urlretrieve(url, str(dest))
    except urllib.error.HTTPError as e:
        raise RuntimeError(f"HTTP error {e.code} when downloading {url}")
    except urllib.error.URLError as e:
        raise RuntimeError(f"URL error {e.reason} when downloading {url}")
    except Exception as e:
        raise RuntimeError(f"Failed to download {url}: {e}")

def auto_download_models(model_dir: str) -> Dict[str, Dict[str, Optional[str]]]:
    """Attempt to automatically download missing model files into model_dir.

    Returns a dict with keys 'prototxt', 'caffemodel', 'pts' holding {'ok': bool, 'url' or 'error': ...}
    """
    model_root = Path(model_dir)
    model_root.mkdir(parents=True, exist_ok=True)

    results: Dict[str, Dict[str, Optional[str]]] = {}

```

```

targets = {
    "prototxt": model_root / "colorization_deploy_v2.prototxt",
    "pts": model_root / "pts_in_hull.npy",
    "caffemodel": model_root / "colorization_release_v2.caffemodel",
}

for key, dest in targets.items():
    if dest.exists():
        results[key] = {"ok": True, "url": str(dest)}
        continue

    urls = DEFAULT_MODEL_URLS.get(key, [])
    ok = False
    last_err = None
    for u in urls:
        try:
            _download_file(u, dest)
            results[key] = {"ok": True, "url": u}
            ok = True
            break
        except Exception as e:
            last_err = str(e)

    if not ok:
        results[key] = {"ok": False, "error": last_err}

return results

def download_from_url(url: str, dest_path: str) -> None:
    """Download a single file from user-supplied URL to dest_path.

    Raises RuntimeError on failure.
    """
    dest = Path(dest_path)
    dest.parent.mkdir(parents=True, exist_ok=True)
    _download_file(url, dest)

def decode_uploaded_image(uploaded_file) -> np.ndarray:
    data = np.asarray(bytearray(uploaded_file.read()), dtype=np.uint8)
    image = cv2.imdecode(data, cv2.IMREAD_COLOR)
    if image is None:
        raise ValueError("Unable to decode the uploaded image.")
    return image

def preprocess_for_unet(image_bgr: np.ndarray, input_size: int = 256) -> Dict[str,

```

```

np.ndarray]:
    rgb = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2RGB)
    gray = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2GRAY)

    rgb_float = rgb.astype(np.float32) / 255.0
    lab = cv2.cvtColor(rgb_float, cv2.COLOR_RGB2LAB)
    l_channel = lab[:, :, 0]

    l_resized = cv2.resize(l_channel, (input_size, input_size),
interpolation=cv2.INTER_AREA)
    l_norm = (l_resized / 50.0) - 1.0

    return {
        "rgb": rgb,
        "gray": gray,
        "l_original": l_channel,
        "l_norm": l_norm,
        "original_size": np.array([rgb.shape[1], rgb.shape[0]], dtype=np.int32),
    }

def postprocess_from_ab(l_original: np.ndarray, ab_pred: np.ndarray) -> np.ndarray:
    h, w = l_original.shape
    ab_resized = cv2.resize(ab_pred, (w, h), interpolation=cv2.INTER_CUBIC)

    lab_out = np.zeros((h, w, 3), dtype=np.float32)
    lab_out[:, :, 0] = l_original
    lab_out[:, :, 1:] = ab_resized

    rgb_out = cv2.cvtColor(lab_out, cv2.COLOR_LAB2RGB)
    rgb_out = np.clip(rgb_out, 0, 1)
    return (rgb_out * 255).astype(np.uint8)

def infer_with_unet(
    model: UNetColorizer, image_bgr: np.ndarray, device: torch.device, return_features:
bool = False
):
    data = preprocess_for_unet(image_bgr)
    l_tensor =
torch.from_numpy(data["l_norm"]).unsqueeze(0).unsqueeze(0).to(device=device,
dtype=torch.float32)

    with torch.no_grad():
        if return_features:
            ab_pred_norm, features = model(l_tensor, return_features=True)
        else:
            ab_pred_norm = model(l_tensor)
            features = None

```

```

ab_pred = ab_pred_norm.squeeze(0).permute(1, 2, 0).cpu().numpy()
ab_pred = ab_pred * 110.0

colorized_rgb = postprocess_from_ab(data["l_original"], ab_pred)

return data["gray"], colorized_rgb, features

def colorize_with_opencv_dnn(
    image_bgr: np.ndarray,
    net: cv2.dnn_Net,
    ab_scale: float = 1.0,
    enhance_contrast: bool = False,
) -> Dict[str, np.ndarray]:
    """OpenCV DNN colorization with corrected LAB pipeline and diagnostics."""
    gray = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2GRAY)
    gray_bgr = cv2.cvtColor(gray, cv2.COLOR_GRAY2BGR)

    # Keep float32 in [0, 1] before LAB conversion.
    img_float = gray_bgr.astype(np.float32) / 255.0
    img_lab = cv2.cvtColor(img_float, cv2.COLOR_BGR2LAB)

    l_original = img_lab[:, :, 0]
    h, w = l_original.shape

    # Model expects 224x224 L channel centered by mean 50.
    l_resized = cv2.resize(l_original, (224, 224), interpolation=cv2.INTER_CUBIC)
    l_centered = l_resized - 50.0
    blob = cv2.dnn.blobFromImage(l_centered)

    net.setInput(blob)
    ab_pred = net.forward()[0, :, :, :].transpose((1, 2, 0))

    # Scale chroma channels for user-controlled color intensity.
    ab_pred = ab_pred * float(ab_scale)
    ab_up = cv2.resize(ab_pred, (w, h), interpolation=cv2.INTER_CUBIC)

    lab_out = np.zeros((h, w, 3), dtype=np.float32)
    lab_out[:, :, 0] = l_original
    lab_out[:, :, 1:] = ab_up

    bgr_out = cv2.cvtColor(lab_out, cv2.COLOR_LAB2BGR)
    bgr_out = np.clip(bgr_out, 0.0, 1.0)

    if enhance_contrast:
        # Mild contrast boost to improve punch without oversmoothing detail.
        out_lab = cv2.cvtColor(bgr_out, cv2.COLOR_BGR2LAB)
        clahe = cv2.createCLAHE(clipLimit=1.6, tileGridSize=(8, 8))
        out_lab[:, :, 0] = clahe.apply((out_lab[:, :, 0] *
255).astype(np.uint8)).astype(np.float32) / 255.0

```



```

    bgr_out = cv2.cvtColor(out_lab, cv2.COLOR_LAB2BGR)
    bgr_out = np.clip(bgr_out, 0.0, 1.0)

    colored_bgr = (bgr_out * 255).astype(np.uint8)
    colored_rgb = cv2.cvtColor(colored_bgr, cv2.COLOR_BGR2RGB)

    # Diagnostics for sepia/low-diversity debugging.
    a_vals = ab_pred[:, :, 0]
    b_vals = ab_pred[:, :, 1]
    stats = {
        "a_min": float(np.min(a_vals)),
        "a_max": float(np.max(a_vals)),
        "a_mean": float(np.mean(a_vals)),
        "a_std": float(np.std(a_vals)),
        "b_min": float(np.min(b_vals)),
        "b_max": float(np.max(b_vals)),
        "b_mean": float(np.mean(b_vals)),
        "b_std": float(np.std(b_vals)),
    }

    gray_lab = cv2.cvtColor(img_float, cv2.COLOR_BGR2LAB)
    color_lab = cv2.cvtColor(bgr_out, cv2.COLOR_BGR2LAB)

    comparison = {
        "gray_a_mean": float(np.mean(gray_lab[:, :, 1])),
        "gray_b_mean": float(np.mean(gray_lab[:, :, 2])),
        "color_a_mean": float(np.mean(color_lab[:, :, 1])),
        "color_b_mean": float(np.mean(color_lab[:, :, 2])),
        "color_a_std": float(np.std(color_lab[:, :, 1])),
        "color_b_std": float(np.std(color_lab[:, :, 2])),
    }

    return {
        "gray": gray,
        "colored_rgb": colored_rgb,
        "ab_pred_small": ab_pred,
        "ab_stats": stats,
        "lab_compare": comparison,
    }

```

Frontend/UI

```
from pathlib import Path
```

```

import cv2
import pandas as pd
import numpy as np
import streamlit as st
import torch

```

```

from model import load_unet_model, summarize_model
from utils import (
    colorize_with_opencv_dnn,
    decode_uploaded_image,
    infer_with_unet,
    load_opencv_dnn_colorization_model,
    models_exist,
    save_uploaded_model_files,
    auto_download_models,
    download_from_url,
)

st.set_page_config(page_title="Deep Learning Image Colorizer", layout="wide")

@st.cache_resource(show_spinner=False)
def get_unet_model(weights_path: str, device_name: str):
    device = torch.device(device_name)
    model, loaded_pretrained = load_unet_model(Path(weights_path), device)
    return model, loaded_pretrained

def get_opencv_net(model_dir: str):
    # No caching here so users can upload files and reload model during runtime
    return load_opencv_dnn_colorization_model(model_dir)

def render_feature_maps(features) -> None:
    if not features:
        return

    st.markdown("##### Intermediate Feature Maps")
    cols = st.columns(len(features))

    for i, (name, fmap) in enumerate(features.items()):
        fmap_2d = fmap[0, 0].detach().cpu().numpy()
        fmap_2d = (fmap_2d - fmap_2d.min()) / (fmap_2d.max() - fmap_2d.min() + 1e-8)
        fmap_u8 = (fmap_2d * 255).astype(np.uint8)

        with cols[i]:
            st.image(fmap_u8, caption=name, width="stretch", clamp=True)

def main() -> None:
    st.title("Deep Learning Image Colorizer")
    st.write(
        "This app colorizes grayscale images in LAB space. "
        "For OpenCV DNN, the L channel is centered by 50 before inference and predicted a,b channels "
    )

```

```

        "are merged back with original L for reconstruction."
    )

    st.markdown("### Model Used")
    model_name = st.radio("Select model", ["OpenCV DNN (Fixed Pipeline)", "U-Net
(PyTorch)"], horizontal=True)

    st.info(
        "DL approach: We convert image RGB->LAB, keep the L channel as input, then
estimate a,b channels "
        "with a CNN model. Finally, L + predicted (a,b) is converted back to RGB."
    )

    with st.sidebar:
        st.markdown("### Color Controls")
        color_intensity = st.slider("Color Intensity (a,b scale)", min_value=0.5, max_value=2.0,
value=1.1, step=0.1)
        enhance_contrast = st.checkbox("Enhance Contrast (CLAHE)", value=False)
        show_hist = st.checkbox("Show a,b histograms", value=True)
        show_lab_compare = st.checkbox("Show LAB comparison", value=True)

    uploaded_file = st.file_uploader("Upload grayscale image", type=["jpg", "jpeg", "png"])

    if uploaded_file is None:
        st.warning("Please upload an image to start.")
        return

    try:
        image_bgr = decode_uploaded_image(uploaded_file)
        preview_gray = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2GRAY)
        preview_gray_rgb = cv2.cvtColor(preview_gray, cv2.COLOR_GRAY2RGB)

        st.markdown("### Input Preview")
        st.image(preview_gray_rgb, caption="Input Grayscale", width="stretch")

        if st.button("Colorize Image", type="primary"):
            with st.spinner("Running deep learning inference..."):
                if model_name == "OpenCV DNN (Fixed Pipeline)":
                    model_dir = Path(__file__).resolve().parent / "models"
                    net = get_opencv_net(str(model_dir))

                    if net is None:
                        st.warning(
                            "OpenCV DNN model files not found in the project's `models/` folder."
                        )

                        # Offer automatic download attempt
                        attempt_auto = st.checkbox("Attempt automatic download of missing model
files", value=True)
                        if attempt_auto:

```

```

with st.spinner("Attempting to download model files..."):
    try:
        results = auto_download_models(str(model_dir))
        st.json(results)
        all_ok = all(results[k]["ok"] for k in ("prototxt", "pts", "caffemodel"))
        if all_ok:
            st.success("All model files downloaded — loading model now...")
            net = get_opencv_net(str(model_dir))
            if net is None:
                st.error("Downloaded files but failed to load the model. The files
may be corrupted.")
            else:
                st.success("Model loaded successfully — continuing inference.")
        else:
            st.error("Auto-download incomplete. See results and either provide a
direct URL or upload missing files.")
    except Exception as e:
        st.error(f'Auto-download failed: {e}')

# If automatic download did not yield a model, offer manual upload and
direct-URL download
if net is None:
    with st.expander("Provide or upload model files (recommended)",
expanded=True):
        st.markdown(
            "Upload the three model files: `colorization_deploy_v2.prototxt`,
`colorization_release_v2.caffemodel`, and `pts_in_hull.npy`."
        )
        proto_up = st.file_uploader("Upload prototxt
(colorization_deploy_v2.prototxt)")
        caff_up = st.file_uploader("Upload caffemodel
(colorization_release_v2.caffemodel)")
        pts_up = st.file_uploader("Upload pts_in_hull.npy")

        if st.button("Save model files to models/ and load"):
            try:
                save_uploaded_model_files(proto_up, caff_up, pts_up,
str(model_dir))

                st.success("Saved files to models/. Attempting to load...")
                net = get_opencv_net(str(model_dir))
                if net is not None:
                    st.success("Model loaded successfully — you can now press
Colorize again.")
                else:
                    st.error("Saved files but failed to load the model. Ensure files are
correct and try again.")
            except Exception as e:
                st.error(f'Failed to save files: {e}')

        st.markdown("---")

```

```

one):")
        st.markdown("Or provide a direct URL to the caffemodel file (if you have
        caff_url = st.text_input("Direct URL to caffemodel (http/https)")
        if st.button("Download caffemodel from URL") and caff_url:
            with st.spinner("Downloading caffemodel from provided URL..."):
                try:
                    download_from_url(caff_url, str(model_dir /
"colorization_release_v2.caffemodel"))
                    st.success("Downloaded caffemodel — attempting to load
model...")

                    net = get_opencv_net(str(model_dir))
                    if net is not None:
                        st.success("Model loaded successfully — you can now press
Colorize again.")
                    else:
                        st.error("Downloaded caffemodel but model failed to load. File
may be invalid.")

                except Exception as e:
                    st.error(f"Failed to download caffemodel: {e}")

            st.info(
                "After placing or uploading the files, press Colorize again to load and run
the model.\n"
                "If you don't have the files, use the U-Net model option instead."
            )
            # Abort inference for this run; user must re-run after adding files
            st.stop()

# At this point net is available
result = colorize_with_opencv_dnn(
    image_bgr=image_bgr,
    net=net,
    ab_scale=float(color_intensity),
    enhance_contrast=enhance_contrast,
)
gray = result["gray"]
colorized_rgb = result["colorized_rgb"]

st.markdown("### a,b Prediction Debug")
st.json(result["ab_stats"])

if show_hist:
    ab_small = result["ab_pred_small"]
    a_flat = ab_small[:, :, 0].reshape(-1)
    b_flat = ab_small[:, :, 1].reshape(-1)

    hist_a, bins_a = np.histogram(a_flat, bins=40)
    hist_b, bins_b = np.histogram(b_flat, bins=40)

    hist_df = pd.DataFrame(

```

```

        {
            "bin_center": (bins_a[:-1] + bins_a[1:]) / 2.0,
            "a_channel": hist_a,
            "b_channel": hist_b,
        }
    )
    st.line_chart(hist_df.set_index("bin_center")[["a_channel", "b_channel"]])

    if show_lab_compare:
        st.markdown("### LAB Comparison (Input Gray vs Colorized)")
        st.json(result["lab_compare"])

    else:
        device_name = "cuda" if torch.cuda.is_available() else "cpu"
        weights_path = Path(__file__).resolve().parent / "models" / "unet_colorizer.pth"
        model, loaded_pretrained = get_unet_model(str(weights_path), device_name)

        gray, colorized_rgb, features = infer_with_unet(
            model=model,
            image_bgr=image_bgr,
            device=torch.device(device_name),
            return_features=True,
        )

        status_msg = (
            "Loaded pre-trained U-Net weights."
            if loaded_pretrained
            else "No U-Net checkpoint found. Running architecture with default
initialization."
        )
        st.caption(status_msg)

        with st.expander("Model Summary", expanded=False):
            st.code(summarize_model(model), language="text")

        with st.expander("Feature Maps", expanded=False):
            render_feature_maps(features)

    st.markdown("### Before vs After")
    col1, col2 = st.columns(2)

    with col1:
        gray_rgb = cv2.cvtColor(gray, cv2.COLOR_GRAY2RGB)
        st.image(gray_rgb, caption="Original Grayscale", width="stretch")

    with col2:
        st.image(colorized_rgb, caption="Colorized Output", width="stretch")

    success, encoded = cv2.imencode(".png", cv2.cvtColor(colorized_rgb,
cv2.COLOR_RGB2BGR))

```

```
    if success:
        st.download_button(
            label="Download Colorized Image",
            data=encoded.tobytes(),
            file_name="colorized_output.png",
            mime="image/png",
        )
    else:
        st.error("Could not prepare image for download.")

except Exception as exc:
    st.error(f"Error: {exc}")
    st.exception(exc)

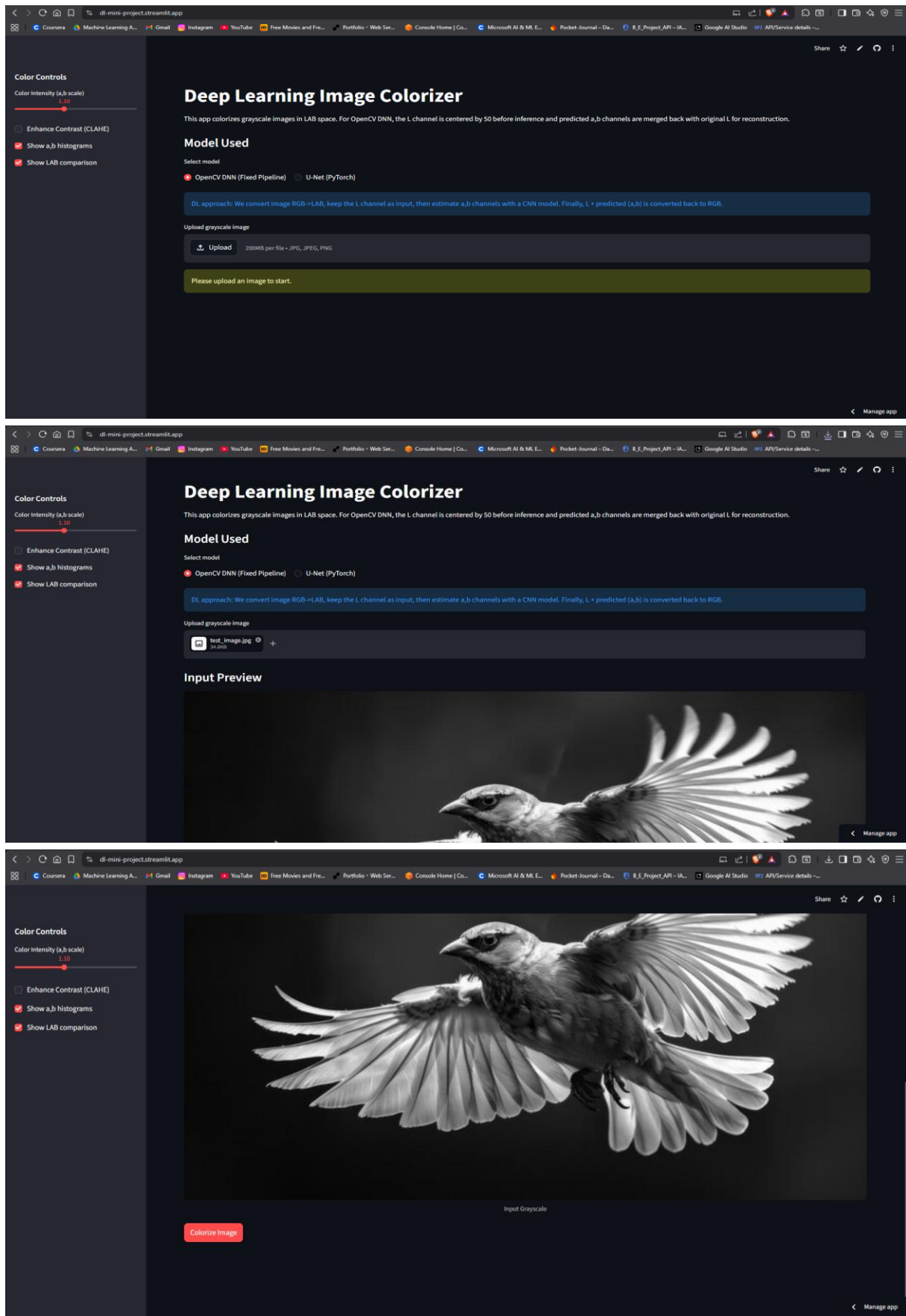
if __name__ == "__main__":
    main()
```

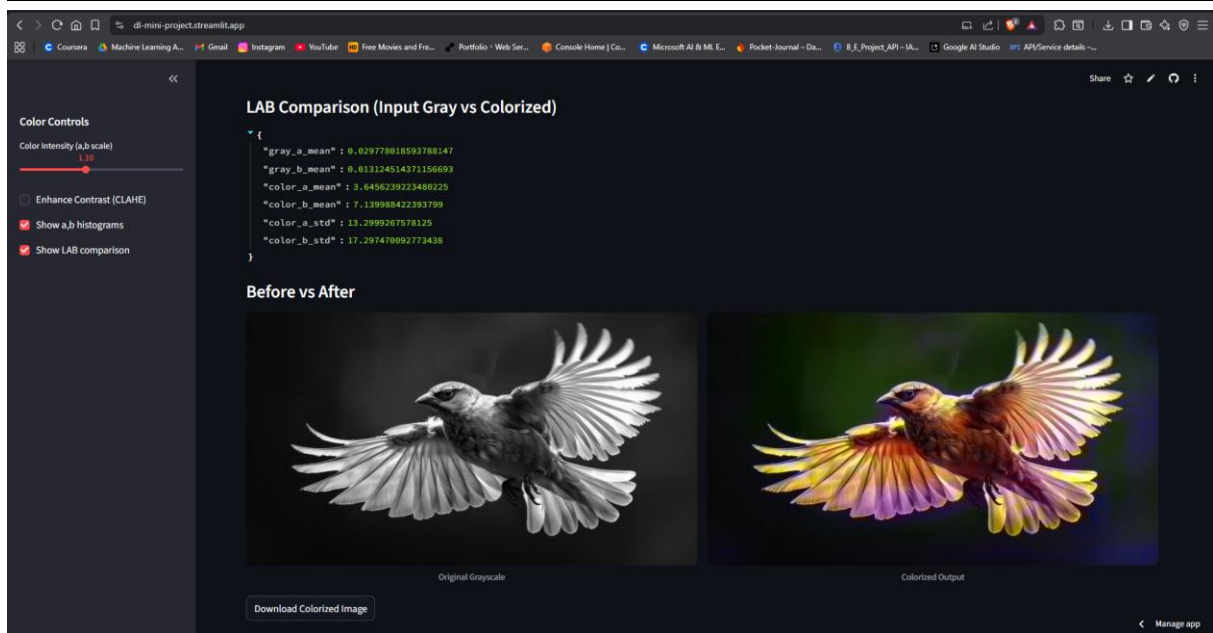
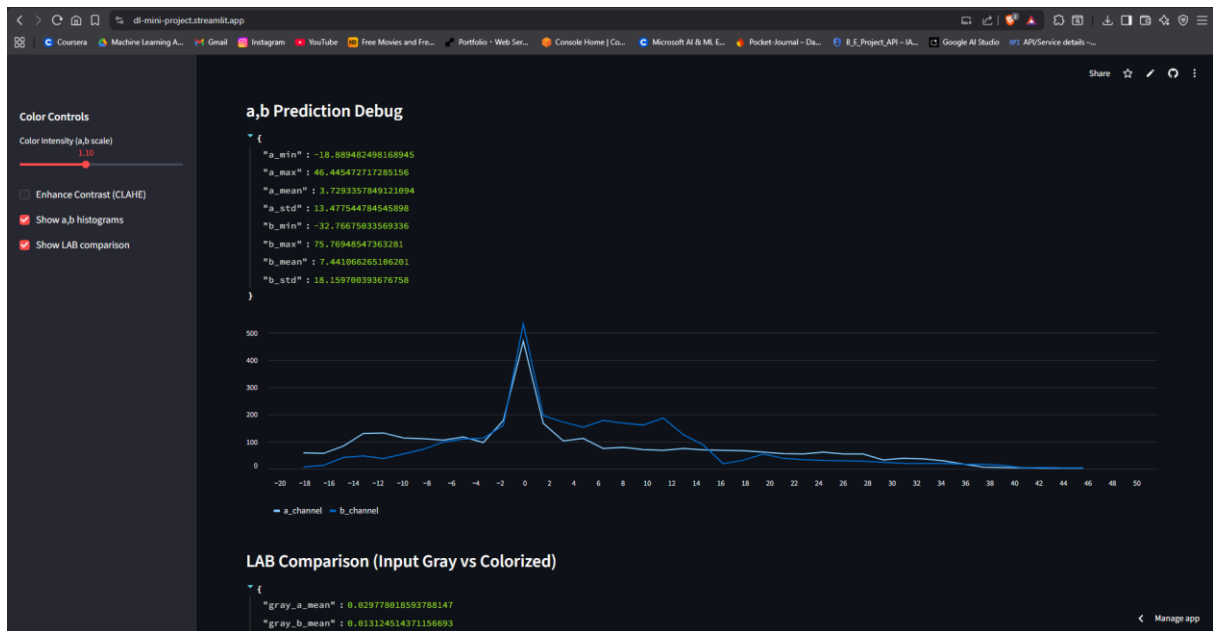
Test Cases

The system is tested for functionality, accuracy, and usability across different modules including input handling, model processing, and output generation.

Module Name	Test Scenario ID	Test Scenario Name	Test Cases	Test Case ID	Test Case Description
Input Module	TS_001	Image Upload	2	TC_001	Verify system accepts valid image formats (JPG, PNG)
				TC_002	Verify error handling for invalid file formats
Processing Module	TS_002	Image Preprocessing	2	TC_003	Verify grayscale conversion of input image
				TC_004	Verify LAB color space conversion
Model Module	TS_003	Model Execution	2	TC_005	Verify OpenCV DNN model loads and runs correctly
				TC_006	Verify U-Net model inference execution
Output Module	TS_004	Colorization Output	2	TC_007	Verify colorized image is generated
				TC_008	Verify output image quality and realism
Validation Module	TS_005	Accuracy Check	2	TC_009	Verify consistency of colorization results
				TC_010	Verify no distortion in output image
UI Module	TS_006	User Interface	2	TC_011	Verify image preview and display functionality
				TC_012	Verify download option for colorized image
System Module	TS_007	Performance	2	TC_013	Verify system handles large images efficiently
				TC_014	Verify faster processing with GPU (if available)

OUTPUT





CONCLUSION

This project successfully implements Huffman Encoding using a hybrid CPU–GPU approach to improve the performance of data compression. By utilizing GPU acceleration through CUDA (Numba), the encoding process is parallelized, reducing execution time compared to traditional CPU-based methods. The system efficiently combines CPU-based operations such as tree construction and decoding with GPU-based parallel encoding, ensuring both accuracy and optimized performance.

Additionally, the project provides a user-friendly interface using Streamlit, allowing users to input data, view compression results, and verify output correctness. The inclusion of a CPU fallback mechanism ensures reliability across systems without GPU support. Overall, the project demonstrates how parallel computing can enhance classical algorithms and offers practical insights into high-performance computing and real-world application development.

REFERENCES: -

Source Code: - <https://dl-mini-project.streamlit.app/>

1. Zhang, R., Isola, P., & Efros, A. A., Colorful Image Colorization, ECCV 2016
2. OpenCV Documentation – Image Colorization using DNN
3. <https://docs.opencv.org/>
4. PyTorch Documentation – <https://pytorch.org/docs/>
5. Streamlit Documentation – <https://docs.streamlit.io/>
6. NumPy Documentation – <https://numpy.org/doc/>
7. Pillow (PIL) Documentation – <https://pillow.readthedocs.io/>
8. GitHub – Image Colorization Projects (OpenCV & Deep Learning)